notes: small CPUs struggle more with false dependencies because the predictors have to be simpler. larger predictors can use tagged associative entries, but this is larger and more power hungry small CPU trend: efficiency cores begin with small CPUs with MDP just to show lookup and power reductions. then present no mdp as icing on top. then talk about whatever the large results are. PGO analysis section showing how it generalises as well as how to play with the threshold to resist larger MDPs

Introduction and motivation: memory disambiguation and MDP most loads independent (constable) trend towards small and efficient cores, they struggle with false deps and unnecessary predictor accesses zero cost hints are helpful for these cases serverless workloads? PND loads (better name?) Contributions

Related work - not sure if differences go here or elsewhere: store distances my work constable - do we want an analysis of how many PNDs load different data/at different address more than 10% (and executed more than 30 times) store set phast

Method: johan stuff

Evaluation: CPI lookups power - runtime dynamic PGO analysis constable comparison store distances

Problems I expect reviewers to have: MDP is such a tiny fraction of core energy impactful gains are with no MDP. but if its so small, why not just have one?

Guidelines for Submission to MICRO 2026

MICRO 2026 Submission #**NaN** – Confidential Draft – Do NOT Distribute!!

Abstract

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

1 Introduction

Modern processors increasingly deploy heterogeneous and efficiency-oriented core designs. Contemporary systems commonly combine large high-performance cores with smaller, energy-efficient out-of-order cores, as seen in ARM big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. Efficiency-focused cores are also widely used in power-constrained systems including smartphones, tablets, smartwatches and other edge systems. These platforms prioritise energy efficiency and area usage while still enabling aggressive speculation and out-of-order (OoO) execution. Despite these constraints, many speculation mechanisms in these cores operate unconditionally on every relevant instruction. To maximise the energy-efficiency of speculative techniques, predictors should ideally only be invoked for instructions that are known to benefit.

Load instructions exemplify this problem. Prior work demonstrates that a considerable portion of load instructions in general-purpose workloads rarely or never exhibit in-flight memory dependencies [**? ? ?**]. Memory dependence prediction (MDP) is a speculative technique used by CPUs to stall loads that have memory dependencies on the appropriate store, while allowing memory-independent loads to schedule as soon as possible [**? ?**]. Despite the ability of most loads to always issue immediately, they all query the MDP on every execution. This is not only inefficient but can also lead to false dependencies due to hash collisions, limiting performance.

We demonstrate that this observation can be exploited to greatly enhance the energy efficiency of the Memory Dependence Predictor (MDP). We implement a profile-guided optimisation technique that labels frequently memory-independent loads by their opcode and simulate the modified binaries using Gem5. These labelled load instructions bypass MDP queries and are always scheduled as early as possible. We show that preventing unnecessary predictions greatly reduces predictor activity while maintaining or even improving IPC.

1.1 Contributions

This paper makes the following contributions:

- Implements a PGO technique to label memory independent loads to the OoO core at no-cost communication or hardware overhead.
- Implements a modified Gem5 to simulate labelled loads against different MDP algorithms and CPU size configurations, as well as extends McPat to provide power usage estimations of the MDP unit.
- Demonstrates that across Spec2017 intspeed:
- An average of 72% (up to 96%) of MDP load queries are either redundant or detrimental to performance,
- Preventing these queries can yield an IPC gain between 0.47% and 3.2% on smaller cores, while maintaining performance on larger cores,
- Average MDP power usage can be reduced between 7% and 42% depending on core size.

2 Background & Related Work

- LSQ, MDP, store sets, PHAST
- old paper
- store distances paper & comparison to us
- constable
- LLVM store queue search skip paper
- LSQ scalability labelling paper

3 Method

- how PGO is commonly used and static analysis doesn't work

4 Evaluation

- experimental design
– cpu size selection reasoning
- IPC
- Lookups
- Energy
- PGO & threshold distance scaling
- Prior work comparison

This section presents the experimental results using our PGO technique. It highlights the magnitude of redundant MDP queries found in general purpose workloads, and the impact on IPC and power usage. Our results demonstrate that a very large portion of predictor activity can be entirely avoided, performance and power usage can be improved, and PND load labels can even act as a partial replacement for the MDP unit altogether, all with zero hardware overhead. Furthermore we examine how well our generated store distance profiles generalise to unseen inputs, and provide a comparison to similar prior work [**?**].

4.1 Experimental Design

5 Threats to Validity

6 Future Work

7 Conclusion

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX